

Les fonctions — Pourquoi et comment ?

Pourquoi utiliser des fonctions ?

- Éviter la répétition : écrire une seule fois, utiliser plusieurs fois
- Modulariser : découper un programme en blocs logiques
- Faciliter la lecture : nommer une action pour la comprendre rapidement
- Faciliter la maintenance : modifier en un seul endroit

Déclaration d'une fonction

```
def nom_de_la_fonction(param1, param2):  
    """Docstring : décrit ce que fait la fonction"""  
    # instructions...  
    return resultat
```

Exemple simple : additionner

```
def addition(a, b):  
    """Retourne la somme de a et b."""  
    somme = a + b  
    return somme  
  
# Appel  
resultat = addition(3, 5)  
print(resultat)  # 8
```

Vocabulaire

- ❑ `def` → mot-clé pour définir
- ❑ `nom_de_la_fonction` → nom explicite (snake_case)
- ❑ `param1, param2` → paramètres (optionnels)
- ❑ `return` → retourne un résultat (optionnel)
- ❑ `docstring` → documentation interne

Les différents types de fonctions personnalisées

Fonction void (sans retour)

```
def afficher_message(nom):  
    """Fonction void : affiche un message"""  
    print(f"Bienvenue {nom} !")  
    # Pas de return  
  
# Appel  
resultat = afficher_message("Alice")  
print(resultat) # Affiche : None
```

Fonction avec retour de valeur

```
def carre(n):  
    """Fonction avec retour : renvoie le carré"""  
    return n ** 2  
  
# Récupération du résultat  
resultat = carre(5)  
print(resultat) # Affiche : 25
```

Fonction avec plusieurs retours possibles

```
def signe(nombre):  
    """Retourne le signe d'un nombre"""  
    if nombre > 0:  
        return "Positif"  
    elif nombre < 0:  
        return "Négatif"  
    else:  
        return "Zéro"  
  
print(signe(10)) # Positif  
print(signe(-5)) # Négatif  
print(signe(0)) # Zéro
```

Fonction avec retour multiple

```
def diviser_et_reste(a, b):  
    """Retourne le quotient et le reste"""  
    quotient = a // b  
    reste = a % b  
    return quotient, reste # Retourne un tuple  
  
q, r = diviser_et_reste(17, 5)  
print(f"Quotient : {q}, Reste : {r}") # Quotient : 3, Reste : 2
```

Tableau récapitulatif

Type de fonction	Mot-clé return	Utilité principale	Exemple
Void	Non (ou return seul)	Effet (affichage, action)	<code>afficher_message()</code>
Retour simple	Oui (une valeur)	Calcul, transformation	<code>carre()</code>
Retours conditionnels	Oui (plusieurs return)	Décisions, cas particuliers	<code>signe()</code>
Retour multiple	Oui (valeurs séparées par virgule)	Renvoyer plusieurs données	<code>diviser_et_reste()</code>

Les différents types de fonctions

Fonctions prédéfinies (built-in)

```
print("Hello")      # Affiche
len("Python")       # Longueur
type(42)             # Type de donnée
int("3")             # Conversion
```

Fonctions de modules

```
import math
print(math.sqrt(16)) # Racine carrée

import random
print(random.randint(1, 10)) # Nombre aléatoire
```

Fonctions anonymes (lambda)

```
carre = lambda x: x ** 2
print(carre(5)) # 25

# Utilisation avec filter(), map(), etc.
nombres = [1, 2, 3, 4]
pairs = list(filter(lambda x: x % 2 == 0, nombres))
print(pairs) # [2, 4]
```

Fonctions récursives

```
def factorielle(n):
    if n <= 1:
        return 1
    return n * factorielle(n - 1)

print(factorielle(5)) # 120
```

Fonctions génératrices (yield)

```
def factorielle(n):
    if n <= 1:
        return 1
    return n * factorielle(n - 1)

print(factorielle(5)) # 120
```

Quand choisir quel type de fonction ?

Tu as un besoin ?

↓

Est-ce une opération simple du quotidien ?

├ Oui → Utilise une fonction prédéfinie

↓

Besoin spécifique mais standard ?

├ Oui → Cherche dans un module existant

↓

Opération très simple et ponctuelle ?

├ Oui → Pense à lambda

↓

Tu vas l'utiliser plusieurs fois ?

├ Oui → Crée une fonction personnalisée

↓

Structure récursive ?

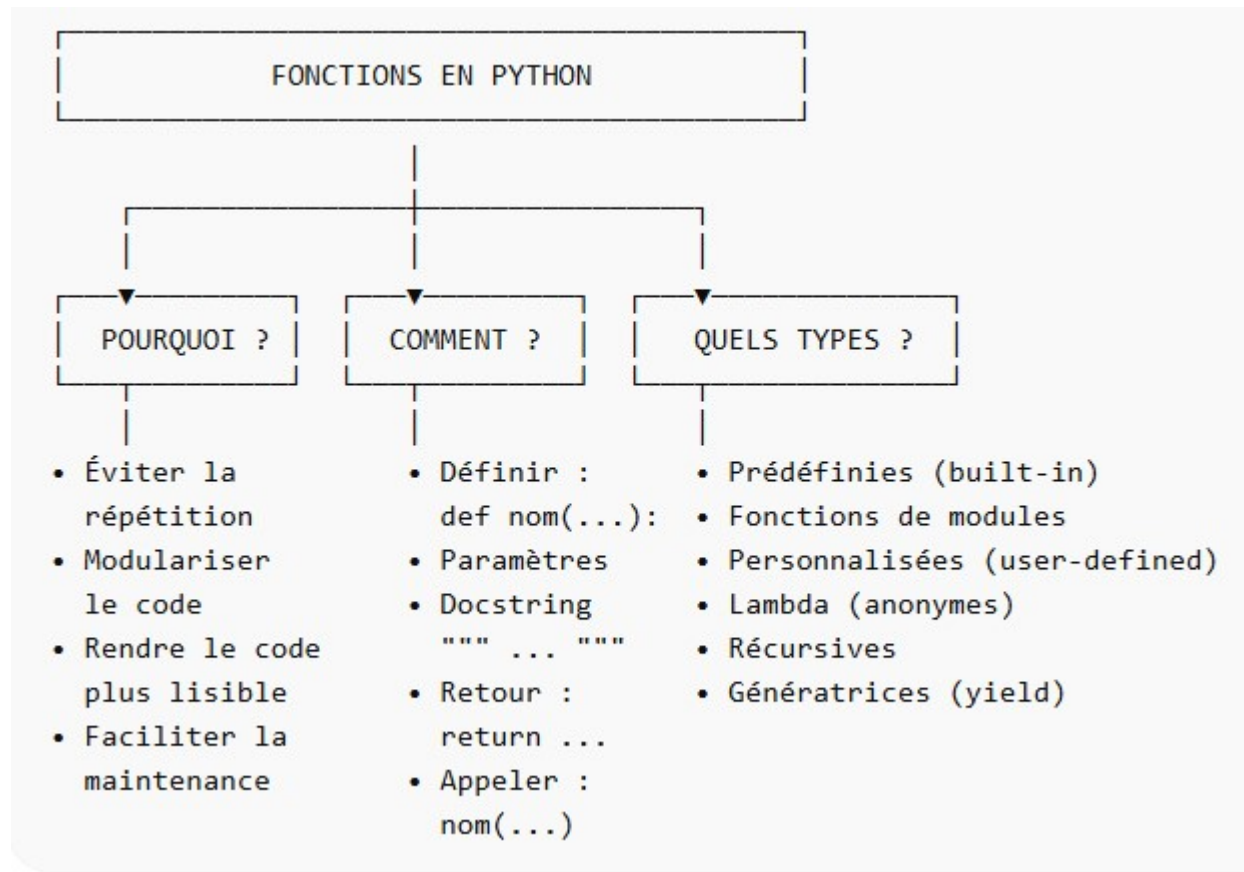
├ Oui → Pense à la récursivité

↓

Gros volume de données ?

├ Oui → Envisage un générateur

Résumé visuel des concepts



Les bonnes pratiques pour écrire des fonctions

Nommage clair et explicite

```
# ❌ À éviter
def f(x):
    return x * 2

# ✅ À privilégier
def doubler(valeur):
    return valeur * 2
```

- ❑ Utilise des verbes pour les actions : `calculer_total()`, `afficher_message()`
- ❑ Sois précis : `get_user_age()` plutôt que `get_data()`
- ❑ snake_case pour Python (mots séparés par des underscores)

Une fonction = une seule responsabilité

```
# ❌ Trop de responsabilités
def traiter_donnees(donnees):
    nettoyer(donnees)
    analyser(donnees)
    sauvegarder(donnees)
    envoyer_email(donnees)

# ✅ Une fonction par tâche
def nettoyer_donnees(donnees): ...
def analyser_donnees(donnees): ...
def sauvegarder_donnees(donnees): ...
def notifier_utilisateur(donnees): ...
```

Toujours documenter (docstring)

```
def calculer_imposition(revenu, tranches):
    """
    Calcule l'impôt à payer selon des tranches d'imposition.

    Args:
        revenu (float): Revenu annuel imposable
        tranches (list): Liste des tranches d'imposition

    Returns:
        float: Montant de l'impôt à payer

    Exemple:
        >>> calculer_imposition(30000, [(0, 0.1), (20000, 0.2)])
        4000.0
    """
    # Code ici
    pass
```


Les bonnes pratiques pour écrire des fonctions

Limiter le nombre de paramètres

```
# ❌ Trop de paramètres (difficile à utiliser)
def creer_utilisateur(nom, prenom, age, email, telephone, adresse, ville, code_postal, pays):
    ...

# ✅ Utiliser un dictionnaire ou une classe
def creer_utilisateur(infos):
    ...

# ou regrouper les paramètres liés
def creer_utilisateur(identite, coordonnees):
    ...
```

Gérer les cas limites

```
def diviser(a, b):
    """
    Divise a par b de manière sécurisée.
    """
    if b == 0:
        return None # ou lever une exception
    return a / b

# Validation des entrées
def traiter_utilisateur(utilisateur):
    if not utilisateur:
        return "Utilisateur invalide"
    if "nom" not in utilisateur:
        return "Nom manquant"
    # Traitement...
```


Exercices

Écris une fonction "saluer" qui prend un paramètre "nom" et affiche "Bonjour, [nom] !"

Tests :

- saluer("Alice") → Bonjour, Alice !
- saluer("Bob") → Bonjour, Bob !

Écris une fonction "celsius_vers_fahrenheit" qui convertit une température de Celsius vers Fahrenheit
(Formule : $^{\circ}\text{C} \times 9/5 + 32 = ^{\circ}\text{F}$)

Tests :

- celsius_vers_fahrenheit(0) → 32.0
- celsius_vers_fahrenheit(100) → 212.0
- celsius_vers_fahrenheit(37) → 98.6

Crée une fonction "calculer" qui prend 3 paramètres : a (nombre), b (nombre), operation (string : "+", "-", "*", "/").
La fonction doit retourner le résultat de l'opération. Gérer la division par zéro en retournant "Erreur: division par zéro".

Tests :

- calculer(10, 5, "+") → 15
- calculer(10, 5, "-") → 5
- calculer(10, 5, "*") → 50
- calculer(10, 5, "/") → 2.0
- calculer(10, 0, "/") → Erreur: division par zéro

Crée une fonction "max_trois" qui prend trois nombres en paramètres et retourne le plus grand des trois (sans utiliser max())

Tests :

- max_trois(5, 8, 3) → 8
- max_trois(10, 10, 5) → 10
- max_trois(-5, -2, -8) → -2

Gestion des Exceptions en Python

Objectif: Maîtriser la gestion des erreurs en Python

```
# Sans gestion d'erreur
age = int(input("Age: ")) # Si l'utilisateur entre "abc" → CRASH

# Avec gestion d'erreur
try:
    age = int(input("Age: "))
    print(f"Vous avez {age} ans")
except ValueError:
    print("Veuillez entrer un nombre valide!")
```

Avantages:

- ❑ Évite les crashes inattendus
- ❑ Améliore l'expérience utilisateur
- ❑ Permet une reprise contrôlée

Syntaxe de base

```
try:
    # Code potentiellement dangereux
    resultat = 10 / 0

except:
    # Code exécuté si une erreur survient
    print("Une erreur est survenue!")
```

```
try:
    nombre = int("abc")
except ValueError:
    print("Conversion impossible!")
```

Types d'Exceptions

Exception	Cause
ValueError	Conversion invalide
ZeroDivisionError	Division par zéro
FileNotFoundError	Fichier inexistant
IndexError	Index hors limites
KeyError	Clé inexistante
TypeError	Type incompatible

Capter des Exceptions Spécifiques

```
try:
    x = int(input("Entrez un nombre: "))
    resultat = 10 / x
    print(f"Résultat: {resultat}")

except ValueError:
    print("Erreur: Ce n'est pas un nombre valide!")

except ZeroDivisionError:
    print("Erreur: Division par zéro impossible!")
```

Gérer plusieurs exceptions

Méthode 1: Plusieurs except

```
try:
    fichier = open("data.txt", "r")
    contenu = fichier.read()
    nombre = int(contenu)

except FileNotFoundError:
    print("Fichier introuvable!")
except ValueError:
    print("Le fichier ne contient pas un nombre valide!")
finally:
    fichier.close()
```

Méthode 2: Grouper les exceptions

```
try:
    resultat = 10 / int("0")
except (ValueError, ZeroDivisionError) as e:
    print(f"Erreur: {e}")
```

else et finally

```
try:
    fichier = open("config.txt", "r")
    contenu = fichier.read()

except FileNotFoundError:
    print("Création du fichier...")
    fichier = open("config.txt", "w")
    fichier.write("default")

else:
    # Exécuté si PAS d'erreur
    print(f"Contenu chargé: {contenu}")

finally:
    # TOUJOURS exécuté
    fichier.close()
    print("Fichier fermé")
```

Résumé:

- ✓ **else** → Si tout s'est bien passé
- ✓ **finally** → Toujours exécuté (nettoyage)

Lever des Exceptions

```
def diviser(a, b):  
    if b == 0:  
        raise ValueError("Le diviseur ne peut pas être zéro!")  
    return a / b  
  
try:  
    resultat = diviser(10, 0)  
except ValueError as e:  
    print(f"Erreur: {e}")
```

Créer une exception personnalisée

```
# Créer une exception personnalisée  
class AgeInvalideError(Exception):  
    pass  
  
def verifier_age(age):  
    if age < 0:  
        raise AgeInvalideError("L'âge ne peut pas être négatif!")  
    if age > 150:  
        raise AgeInvalideError("Âge improbable!")  
    return f"Âge valide: {age}"
```

Exemple Pratique 1

```
def lire_fichier_securise(nom_fichier):  
    """Lecture sécurisée d'un fichier"""  
    try:  
        with open(nom_fichier, 'r', encoding='utf-8') as f:  
            return f.read()  
  
    except FileNotFoundError:  
        print(f"Fichier '{nom_fichier}' introuvable")  
        return None  
  
    except PermissionError:  
        print(f"Permission refusée pour '{nom_fichier}'")  
        return None  
  
    except Exception as e:  
        print(f"Erreur inattendue: {e}")  
        return None  
  
# Test  
contenu = lire_fichier_securise("data.txt")  
if contenu:  
    print(f"Contenu: {contenu[:100]}...")
```


Exemple Pratique 2

```
def saisir_entier(message, min_val=None, max_val=None):
    """Saisie robuste d'un entier avec validation"""
    while True:
        try:
            valeur = int(input(message))

            # Validation des bornes
            if min_val is not None and valeur < min_val:
                print(f"Valeur trop petite (min: {min_val})")
                continue

            if max_val is not None and valeur > max_val:
                print(f"Valeur trop grande (max: {max_val})")
                continue

            return valeur

        except ValueError:
            print("Veuillez entrer un nombre valide!")

# Utilisation
age = saisir_entier("Entrez votre âge: ", 0, 120)
print(f"Âge enregistré: {age} ans")
```

Bonnes Pratiques

```
# À ÉVITER
try:
    # Code...
except: # Capture TOUTES les erreurs (même Ctrl+C!)
    pass # Silence total - TRÈS DANGEREUX

# À PRIVILÉGER
try:
    # Code...
except ValueError as e: # Spécifique
    logger.error(f"Erreur de valeur: {e}")
    raise # Relever si nécessaire
```

Règles d'or:

- Capturer uniquement ce que vous savez gérer
- Toujours spécifier le type d'exception
- Ne pas faire de `except: pass` silencieux
- Utiliser `finally` pour le nettoyage
- Documenter les exceptions dans les fonctions